Lepage analysis: a Renormalization Approach to Solve Schrödinger Equation

Theoretical and Numerical Aspects of Nuclear Physics

Caterina Vagnoni

Table of contents

Introduction

This work is inspired from the lectures [1] of G. P. Lepage about how to renormalize the Schrödinger equation.

The main idea is that renormalization tecniques, developed in a QFT framework, can be also exploited in non-relativistic quantum mechanics.

Conventions: $\hbar = 1, \quad m = 1, \quad q = 1, \quad Z = 1.$

Physical system

The physical model we are going to consider is given by a one-particle atom with:

- Coulomb interaction (long-range)

$$V_{\text{Coulomb}}(r) = -\frac{1}{r} \tag{1}$$

- Additional short-range interaction

$$V_s(r) = -\frac{e^{-r}}{r} \tag{2}$$

The total real potential is $V_{\text{real}} = V_{\text{Coulomb}} + V_s$.

Procedure

Renormalization theory tells us that we can model low-energy data with arbitrary precision, even if the real short-range potential is unknown.

Here the followed procedure is:

- Generate data from the real theory
- Generate data by approximating $V_s(r)$ to a tuned delta function (diverging contact term)
- Generate data from an *effective theory* obtained using renormalization techniques
- Compare the results

Renormalization technique

Two main steps:

1. Introduce an energy cutoff Λ at which the unknown physics effects become important, to soften short-distance behaviour:
   - Exclude high-momentum states, sensitive to the unknown short-distance dynamics
   - Make all interactions regular at r=0

2. Add to the lagrangian local (only a finite number of corrections needed to achieve any level of precision) interactions that mimic the effects of short-range physics excluded by the cutoff: these are systematically suppressed by powers of the cutoff.

Solving Schrödinger equation

The problem is to solve is the radial Schrödinger equation, independent on time:

$$-\frac{1}{r^2}\frac{d^2\psi(r)}{d\,r^2} + V(r)\,\psi(r) = E\,\psi(r) \tag{3}$$

The code uses a well-known numerical method from Numerov inspired from [2] to solve differential equations.

Numerov method

The starting point is an equation of the form:

$$\frac{d^2y}{dx^2} = -g(x)\,y(x) + s(x)\,. \tag{4}$$

In this case $s(x) = 0$.
Using Taylor expansions at the points $x \pm dx$, one can arrive to the *Numerov formula*, which is a discrete result useful for numerical computations:

$$y_{n+1} = \frac{(12 - 10f_n)y_n - f_{n-1}y_{n-1}}{f_{n+1}} \tag{5}$$

where $f_n \equiv 1 + g_n \frac{dx^2}{12}$.

Numerov method applied to Schrödinger equation

A complication that discourages the naive application of the Numerov method to Schrödinger equation is the singular behaviour of the potential at $r \to 0$.

A better numerical analysis can be done by probing values of r with variable step grid, with more points at low values of r.

However, the Numerov method requires a constant-step grid.

Solution: introduce an auxiliary variable with constant-step grid.

Logarithmic grid

Auxiliary variable: $x(r) = \ln r$.

Constant step: $dx = \frac{dr}{r}$

```
def grid(n_grid, x_min, dx):

    x = np.linspace(x_min, x_min + ((n_grid - 1) * dx), n_grid)
    x = np.append(x, x[n_grid - 1] + dx)

    #generate r, sqrt_r, and r^2 (logarithmic grid)
    r = np.exp(x)
    sqrt_r = np.sqrt(r)
    r2 = np.power(r, 2)
    for i in range(n_grid):
        if r[i]>100:
            i_100 = i  #index corresponding to r = 100 to compute phase shift
            break

    return r, sqrt_r, r2, i_100
```

Adjusting the Schrödinger equation

Substituting $\psi(r) = y(r)$ with $y(r(x))$ the equation gets an additional term that goes as the first derivative of y.
To go back to an equation with a suitable form for the Numerov method, redefine:

$$\psi(r) = \sqrt{r}\, y(r(x))\,. \tag{6}$$

With this substitution, we obtain the equation

$$\frac{d^2y}{dx^2} = -\left(2r^2(E - V(r(x)) - \frac{1}{4}\right)\, y(x) \tag{7}$$

which is solvable with Numerov.

Generalization with non-zero angular momentum

So far a quantum number $l = 0$ has been assumed, i.e. one consider only spherically symmetric solutions (S-waves).

$$V(r) \to V_{eff}(r) = -\frac{1}{r} + \frac{l(l+1)}{2r^2} \tag{8}$$

$$\frac{d^2y}{dx^2} = -\left(2r^2(E - V(r(x)) - \left(l + \frac{1}{2}\right)^2\right) y(x) \tag{9}$$

However, in this analysis, only the case $l = 0$ will be addressed.

How to solve Schrödinger equation: code

Function that numerically solves the equation, inspired by [2]:

```
def solve_schrodinger(n_grid, dx, v_potential, r2, r, sqrt_r, n, l, i_100)
```

Arguments:

- `n_grid` = number of points in the grid
- `dx` = constant grid spacing (dx)
- `v_potential` = potential to investigate
- `r2`, `r`, `sqrt_r` = values of r^2, r and $\sqrt{r}$ in the grid
- `n`, `l` = quantum numbers
- `i_100` = index of the grid at which r becomes greater than 100, used to calculate the phase shift.

Energy eigenvalues and eigenfunctions

One first needs to find the eigenvalues E, which appear in the function g of the Numerov algorithm but are not known.

To do so, the code uses the following procedure:

- Take an initial guess for the eigenvalue as the avarage of the minimum and maximum value of the potential in the range of the grid

```
1        lpar = (l + 0.5) * (l + 0.5)
2    #initial lower and upper bounds to the eigenvalue
3    e_up = v_potential[n_grid] #bound state has negative energy
4    e_low = e_up
5
6    #min energy is greater than the min of the effective potential
7    for j in range(0, n_grid + 1):
8            e_low = np.minimum(e_low, lpar / (2 * r2[j] ) + v_potential[j])
9
10   if (e_up - e_low < eps):
11       print("error in solving schrodinger: e_up and e_low coincide",
12             file = sys.stderr)
13       sys.exit(1)
14
15   e = (e_low + e_up) * 0.5 #first rough estimate
```

Energy eigenvalues and eigenfunctions

- Start a `while` loop to determine the eigenvalue. The first part of the loop determines the point of classical inversion for the Schrödinger equation, $g(x_{cl}) = 0$.

```
1  while i  < n_iter and np.absolute(de) > eps:
2
3      g[0] = ddx12 * ((2 * r2[0] * (v_potential[0] - e)) + sqlhf)
4      for j in range(1, n_grid + 1):
5          g[j] = ddx12 * ((2 * r2[j] * (v_potential[j] - e)) + sqlhf)
6
7          #take the index of classical inversion
8          if np.sign(g[j]) != np.sign(g[j - 1]):
9              inv_point = j
10
11     if inv_point < 0 or inv_point >= n_grid - 2:
12         print(f"{inv_point:4d} {n_grid:4d} {n}")
13         print("error in solving schrodinger: last change of sign too far",
14         file = sys.stderr)
15         sys.exit(1)
```

The importance of the classical inversion point is due to the fact that the first derivative of the function will be continuous there only if we have reached an eigenvalue of the energy.

Energy eigenvalues and eigenfunctions

- Use Numerov formula to calculate the eigenfunction, count the number of points where it goes to zero and compare with the theoretical number of nodes. This is a check to see if the eigenvalue guess needs to be lowered or increased.

```
    f = 1 - g #numerov f
    y = np.zeros(n_grid + 1) #wavefunction initialization
    nodes = n - l - 1
    #wavefunction in the first two points b. c.
    y[0] = 1E-12
    y[1] = 1E-5
    #outward integration with node counting
    n_cross = 0
    for j in range(1, inv_point): #numerov formula
        y[j + 1] = ((12. - f[j] * 10.) * y[j] - (f[j - 1] * y[j - 1])) / f[j + 1]
        if np.sign(y[j]) != np.sign(y[j + 1]):
            n_cross += 1
    #check the number of crossings
    if (n_cross != nodes):
        #incorrect number of nodes, adjusting eigenvalue
        if (n_cross > nodes): #means my eigenvalue is too high for n
                e_up = e
        else:
            e_low = e
        e = (e_up + e_low) * 0.5
    i += 1 #and start the loop from the beginning with different e
```

Energy eigenvalues and eigenfunctions

- If instead the number of nodes coincides with the counted roots, the code proceeds with inward integration

```
    y[n_grid] = 0
    y[n_grid - 1] = dx

    for j in range(n_grid - 1, inv_point, -1):
        y[j - 1] = ((12. - f[j] * 10.) * y[j] - (f[j + 1] * y[j + 1])) / f[j - 1]
        if (y[j - 1] > 1E10): #rescaling to avoid numerical overflow
            for m in range(n_grid, j - 2, -1):
                y[m] = y[m] / y[j - 1]
```

- rescaling of the function at the matching point (classical inversion)

```
    scale_factor /= y[inv_point] #rescale the function to match at x_cl
    for j in range(inv_point, n_grid + 1):
        y[j] *= scale_factor

    norm = 0.
    for j in range(0, n_grid + 1):
        norm += y[j] * y[j] * r2[j] * dx
    norm = np.sqrt(norm)

    for j in range(0, n_grid + 1):
        y[j] /= norm
```

Energy eigenvalues and eigenfunctions

- I have the correct number of nodes so e is near an eigenvalue, but needs to be refined.

```
    j = inv_point
    y_cusp = (y[j - 1] * f[j - 1] + f[j + 1] * y[j + 1] + f[j] * 10. * y[j]) / 12.
    #ycusp is the value predicted by the Numerovs method using xcl as central point
    #should be equal to y[j] if the wavefx is smooth (correct eigenvalue)
    #df = fcusp - f(inv_point) with f_cusp allows the fx to satisfy numerov at x_cl
    df_cusp = f[j] * ((y[j] / y_cusp) - 1.)
    # eigenvalue update using perturbation theory
    de = df_cusp / ddx12 * y_cusp * y_cusp * dx / 2
    if (de > 0.):
        e_low = e
    if (de < 0.):
        e_up = e
    e = e + de
    i += 1 #restart cycle
```

The non-smoothness of the wavefunction can be seen as if we are considering a correct eigenvalue for a potential modified with a delta function at $x = x_{cl}$. The change in the potential is related, through perturbation theory, to the difference between the current estimate and the true eigenvalue $dV \sim df \sim dE$ [2].

Energy eigenvalues and eigenfunctions

- The `while` cycle ends when the main index reaches maximum iteration number `n_iter`.
  If dE is too big the cycle did not converge to a nice eigenvalue and we have to change the iteration parameters.
- If the cycle converges, the phase shift is calculated at $r = 100$:

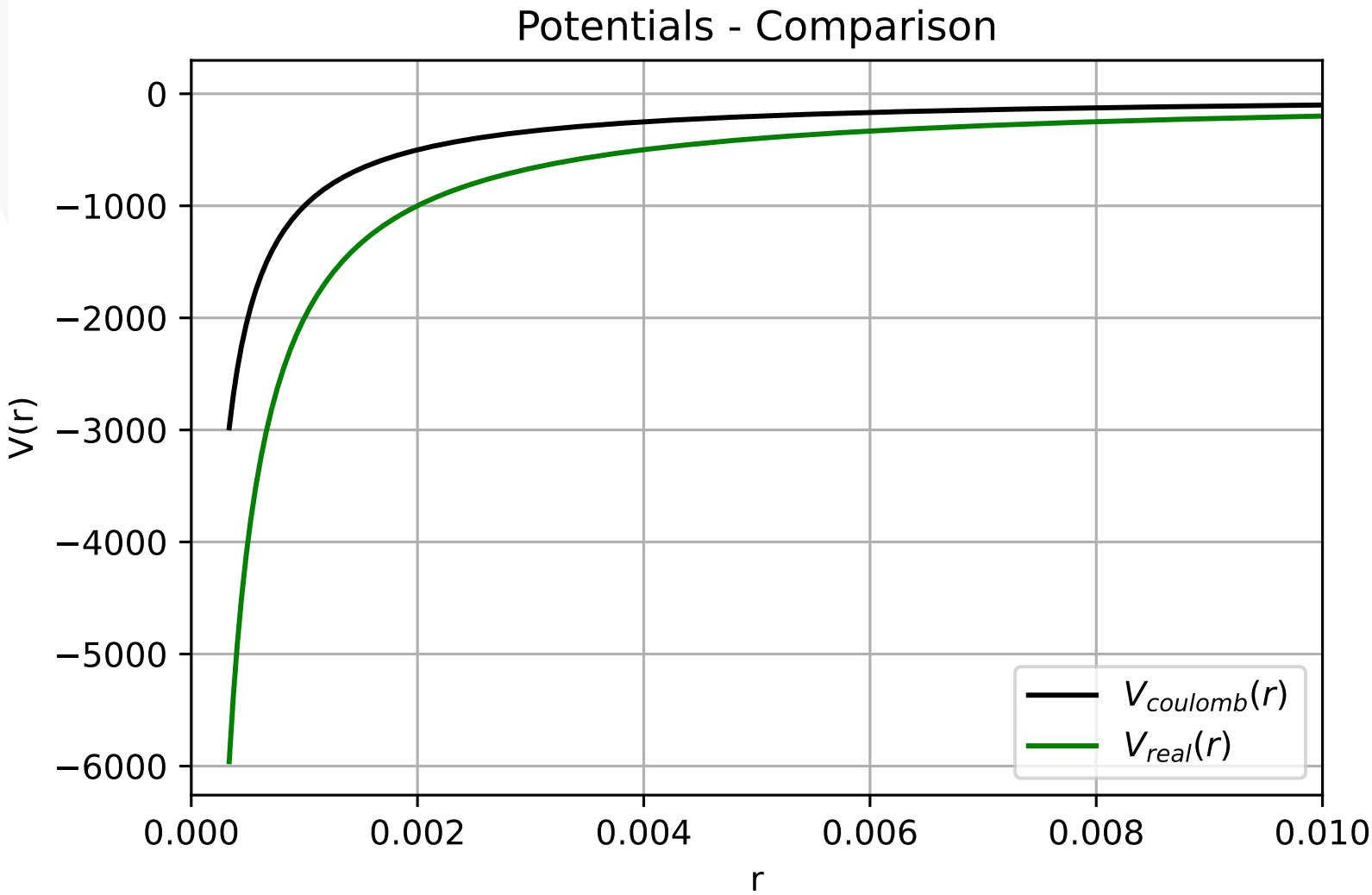
$$y(r) \sim \sin(\sqrt{2mE}\, r + \varphi) \implies \varphi = \arcsin(r) - \sqrt{2mE}\, r \tag{10}$$

```
1  phase_shift = (np.arcsin(y[i_100]) - np.sqrt(np.absolute(e)) * r[i_100]
2      + (l * np.pi)/2)
```

- The function returns:

```
1      return e, y, phase_shift
```

Potentials


Potentials - Comparison
0
−1000
−2000
−3000
−4000
−5000
−6000
V(r)
0.000
0.002
0.004
0.006
0.008
0.010
r
$V_{coulomb}(r)$
$V_{real}(r)$

Potential analysis

The function

```
analysis(n_max, pot_name, e_n, p_n, y_n, n_grid, dx, v_potential, r2 , r, sqrt_r, i_100)
```

implements `solve_schrodinger` to each energy level up to `n_max`.

It also prints the results:

- plots of the eigenfunctions
- table of energy eigenvalues
- table of phase shifts

and collect the corresponding data

```
1     return e_n, y_n, p_n
```

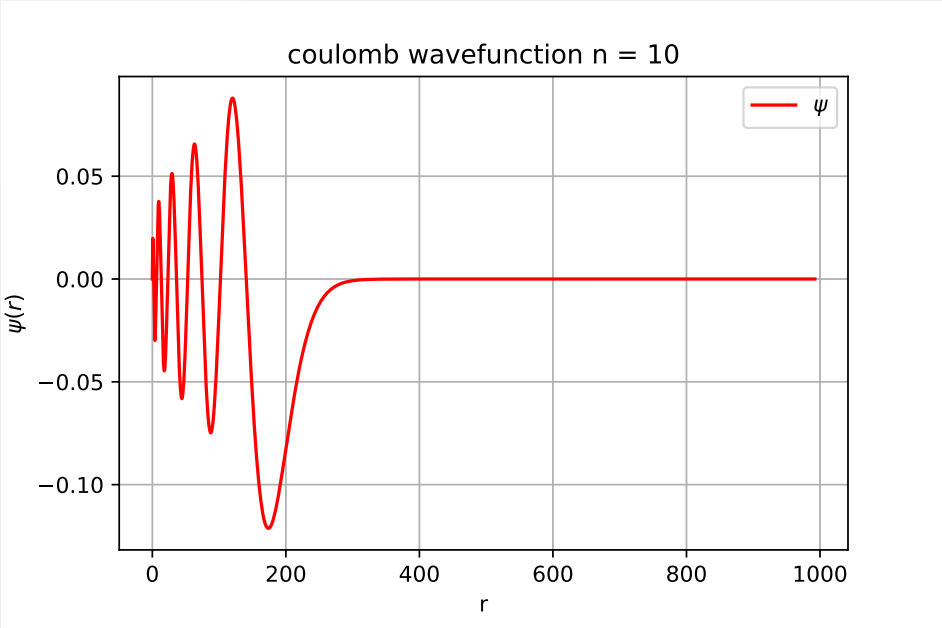


Figure: Example of the plot of an eigenfunction ($n = 10$ - Coulomb potential).

Coulomb potential analysis: results

n	E	phase shift
1	-0.499333	-100.417
2	-0.124917	-50.2253
3	-0.0555308	-33.4873
4	-0.0312396	-25.1168
5	-0.0199947	-20.0941
6	-0.0138858	-16.747
7	-0.0102021	-14.3436
8	-0.0078112	-12.5724
9	-0.00617192	-11.1715
10	-0.00499933	-10.0489
11	-0.00413173	-9.13214
12	-0.00347184	-8.36956
13	-0.00295828	-7.72507
14	-0.00255078	-7.17306
15	-0.00222203	-6.69485
16	-0.00195296	-6.27651
17	-0.00172997	-5.90743
18	-0.0015431	-5.57938
19	-0.00138495	-5.28586
20	-0.00124987	-5.02158

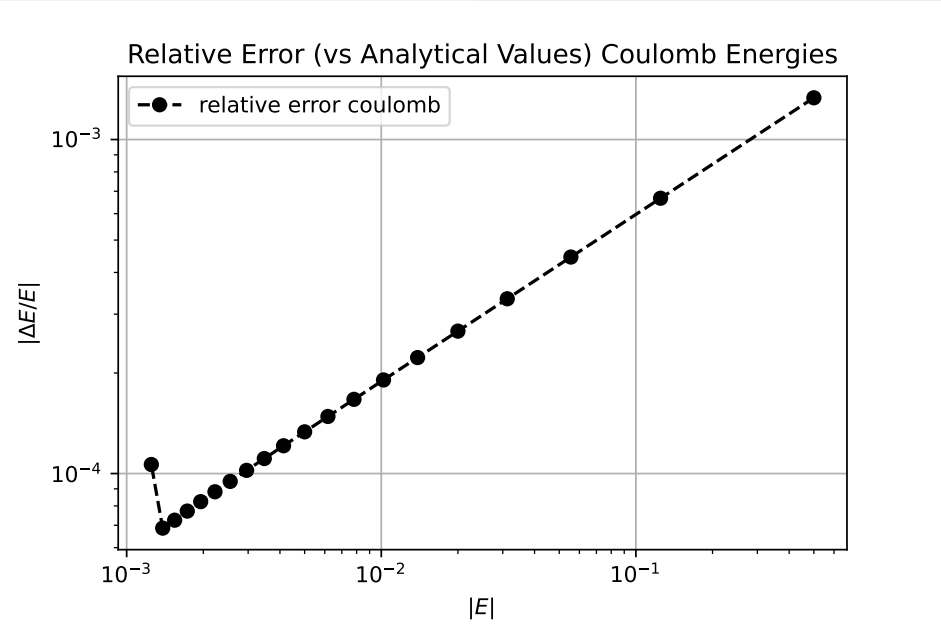


Figure: The numerical solution is more precise for low binding energy states.

Real potential analysis: results

n	E	phase shift
1	-1.28242	-160.927
2	-0.18302	-60.7941
3	-0.0703051	-37.6796
4	-0.0371227	-27.3799
5	-0.0229138	-21.511
6	-0.015542	-17.7166
7	-0.0112309	-15.0536
8	-0.00849356	-13.1115
9	-0.00664752	-11.5873
10	-0.00534395	-10.3829
11	-0.00438939	-9.40825
12	-0.0036695	-8.60232
13	-0.00311322	-7.92413
14	-0.00267448	-7.34527
15	-0.00232235	-6.84527
16	-0.00203545	-6.40901
17	-0.00179861	-6.02502
18	-0.00160082	-5.68441
19	-0.00143395	-5.38024
20	-0.00129187	-5.10692

It is clear that the short range term influences more high-energy states, while at higher `n` (lower binding energy) the discrepance between Coulomb and full potential results are much more negligible.

These are the synthetic data that one aims to reproduce with an effective theory.

Reproduce the full potential results: naive approach

Approximate the potential with a point-like interaction like:

$$V(r) = -\frac{1}{r} + c\,\delta^{(3)}(\mathbf{r}) \tag{11}$$

and tune c in order to best reproduce the data generated with the real potential.
Associated energy eigenvalues:

$$E_n^{\delta} = -\frac{1}{2n^2} + c\,\frac{\delta_{l,0}}{\sqrt{\pi}\,n^3} \tag{12}$$

Comparing $E^{\delta}_{n_{max}}$ with the numerical eigenvalue for the full potential $\Longrightarrow$ `c = -0.593631734330854`.

Reproduce the full potential results: naive approach

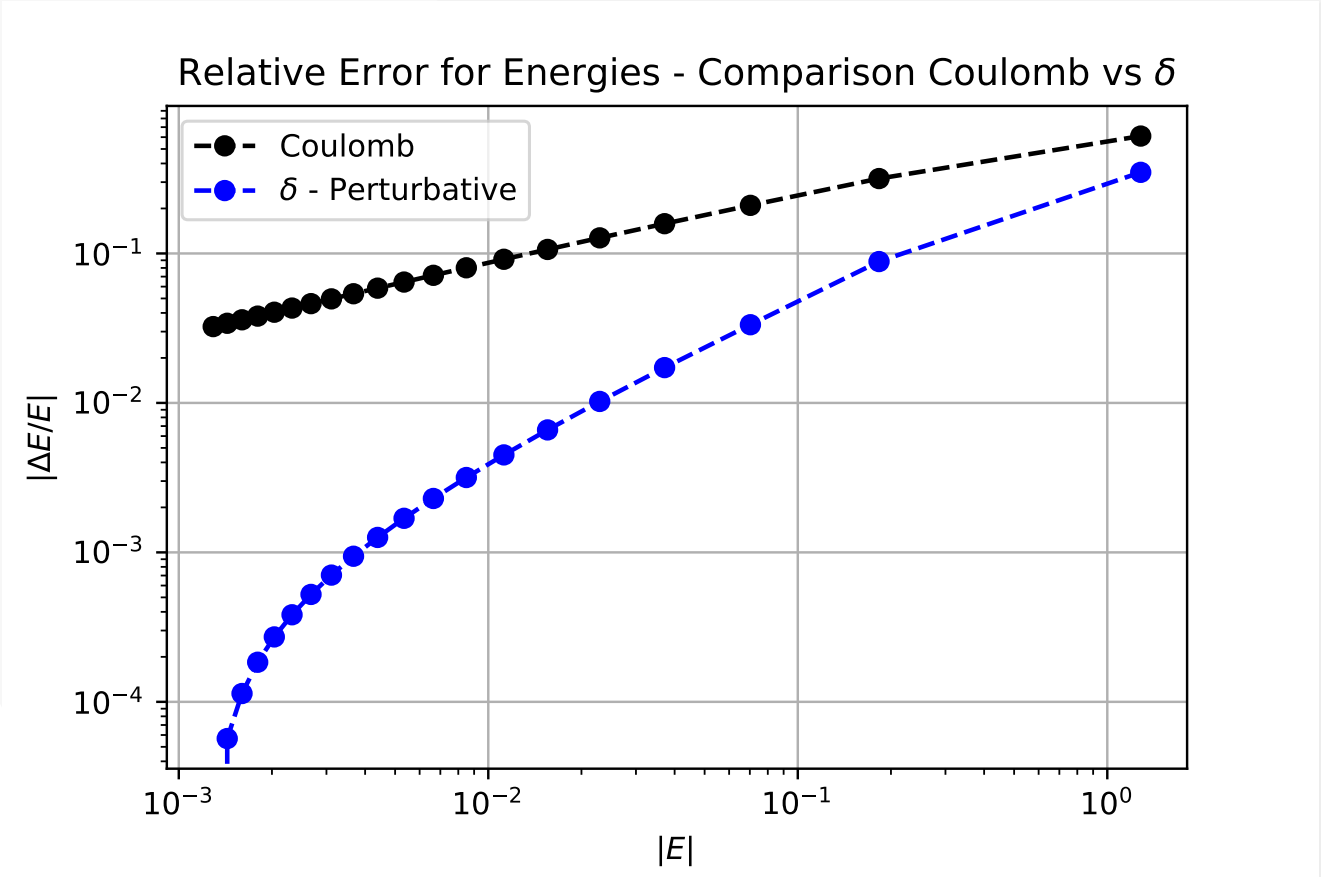


Figure: Introducing the δ function to the Coulomb potential, one already gets a huge improvement on the estimated eigenvalues with low binding energy.

Limitations of the δ function approach

The extreme singularity of the δ is an obstacle to deeper precision:

- Corrections beyond the first order in perturbation theory give diverging contribution
- Adding correction terms to the potential depending on the derivatives of the δ leads to a diverging contribution even at first order. These corrections emerge if more terms of the Taylor expansion of the potential's Fourier transform are not neglected.

$$V_{eff}(\mathbf{r}) = -\frac{1}{r} + c\,\delta^3(\mathbf{r}) + d_1\nabla^2\delta^3(\mathbf{r}) + \cdots + g\nabla^n\delta^3(\mathbf{r}) + \ldots \qquad (13)$$

Reproduce the full potential results: renormalization

Introduction of the cutoff to regulate Coulomb singularity:

$$-\frac{1}{r} \to -\frac{1}{r}\mathrm{erf}\left(\frac{r}{\sqrt{2}a}\right), \quad a = \frac{1}{\Lambda} \tag{14}$$

Adding the local terms to the potential:

$$V_{eff}(\mathbf{r}) = -\frac{1}{r}\mathrm{erf}\left(\frac{r}{\sqrt{2}a}\right) + ca^2\delta_a^3(\mathbf{r}) + d_1a^4\nabla^2\delta_a^3(\mathbf{r}) + \cdots + ga^{n+2}\nabla^n\delta_a^3(\mathbf{r}) + \ldots \tag{15}$$

Where $\delta_a^3(\mathbf{r})$ is a smeared δ function:

$$\delta_a^3(\mathbf{r}) = \frac{e^{-r/a}}{8\pi a^3}, \quad \nabla^2\delta_a^3(\mathbf{r}) = \frac{e^{-r/a}}{8\pi r a^4}\left(\frac{r}{a} - 2\right) \tag{16}$$

Reproduce the full potential results: renormalization

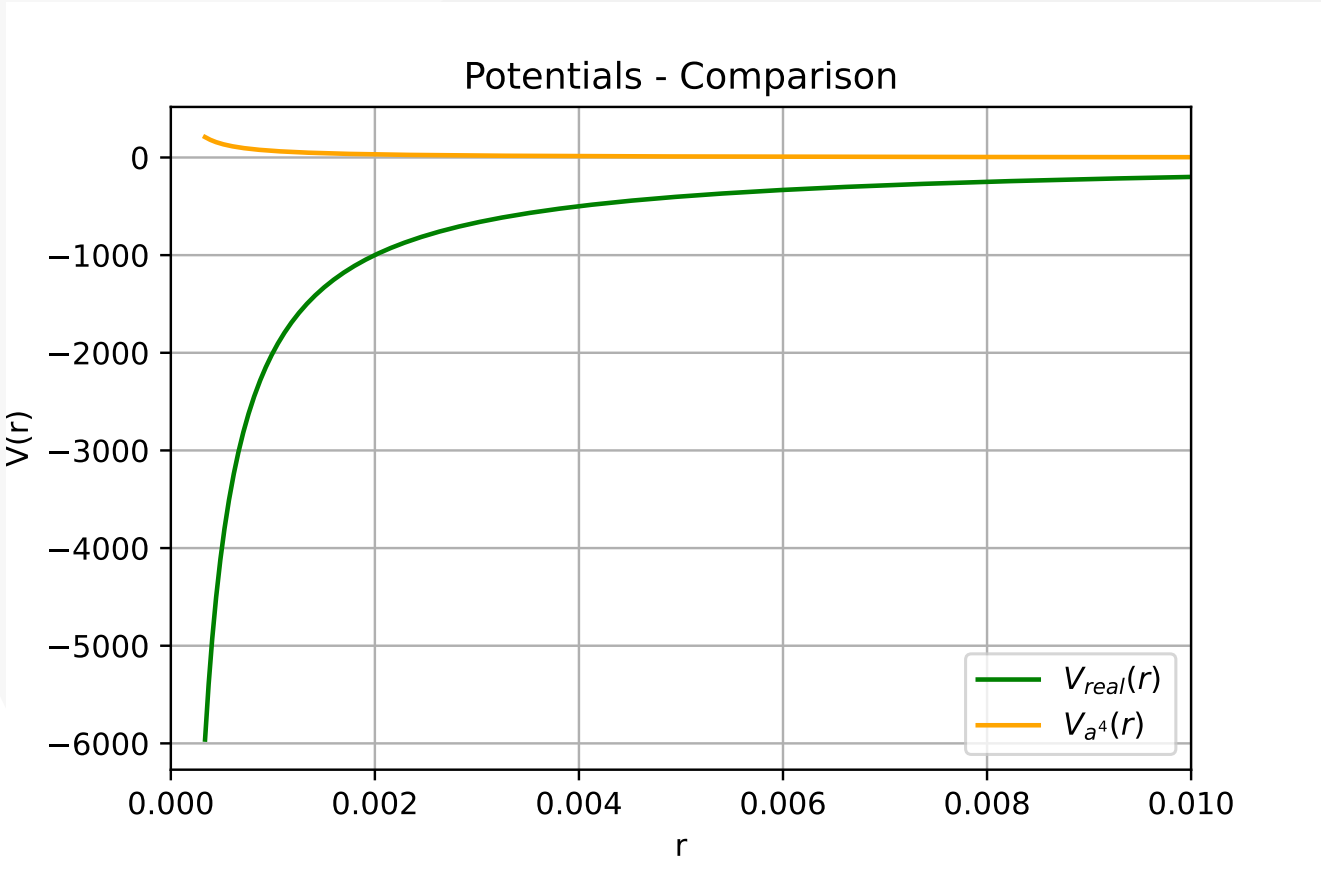


Figure: The potential is regularized at $r = 0$ but goes asymptotically as the real and Coulomb potential.

Reproduce the full potential results: renormalization

To fix the coupling constants, compare the phase shifts of the states with lower binding energy.
For example, with an a^2 truncation:

```
from scipy.optimize import minimize

def cost_function_1(c, a, n_grid, dx, r2, r, sqtr_r, n_max):
v_potential = init_potential_effective(r, a, c, 0) #imposing d_1 = 0
e, y, phase_shift =
    solve_schrodinger(n_grid, dx, v_potential, r2, r, sqrt_r, n_max, 0, i_100)
return np.absolute(ph_shift_real[-1] - phase_shift) #index -1 for last element

result1 = minimize(
cost_function_1,
c_a2,
method='nelder-mead',
args=(a, n_grid, dx, r2, r, sqrt_r, n_max),
options={'maxiter': 10000, 'maxfev': 10000, 'xatol': 1E-8, 'disp': False}

c_a2 = result1.x
```

Results of the effective theory

Results for $a = 1$:

	a^2 theory	a^4 theory
c	-76.13981762	-75.08693367326089
d_1		-1.052883944446429

First 10 data of the effective theory at a^2 and a^4 orders:

n	E	phase shift
1	-1.02978	-144.206
2	-0.203812	-64.1546
3	-0.0712741	-37.9384
4	-0.0372966	-27.444
5	-0.0229627	-21.534
6	-0.0155597	-17.7266
7	-0.0112384	-15.0587
8	-0.00849711	-13.1143
9	-0.00664934	-11.5888
10	-0.00534494	-10.3838

n	E	phase shift
1	-1.02976	-144.205
2	-0.204393	-64.2459
3	-0.0712975	-37.9446
4	-0.0373007	-27.4455
5	-0.0229639	-21.5345
6	-0.0155601	-17.7269
7	-0.0112386	-15.0588
8	-0.00849719	-13.1144
9	-0.00664938	-11.5888
10	-0.00534497	-10.3838

Results of the effective theory

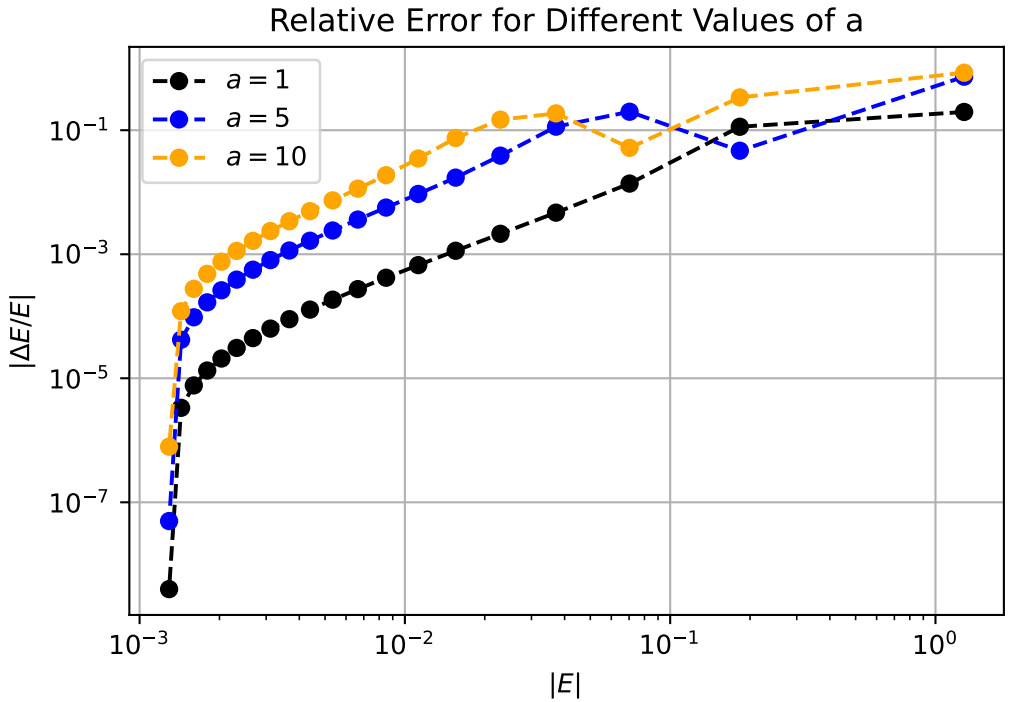
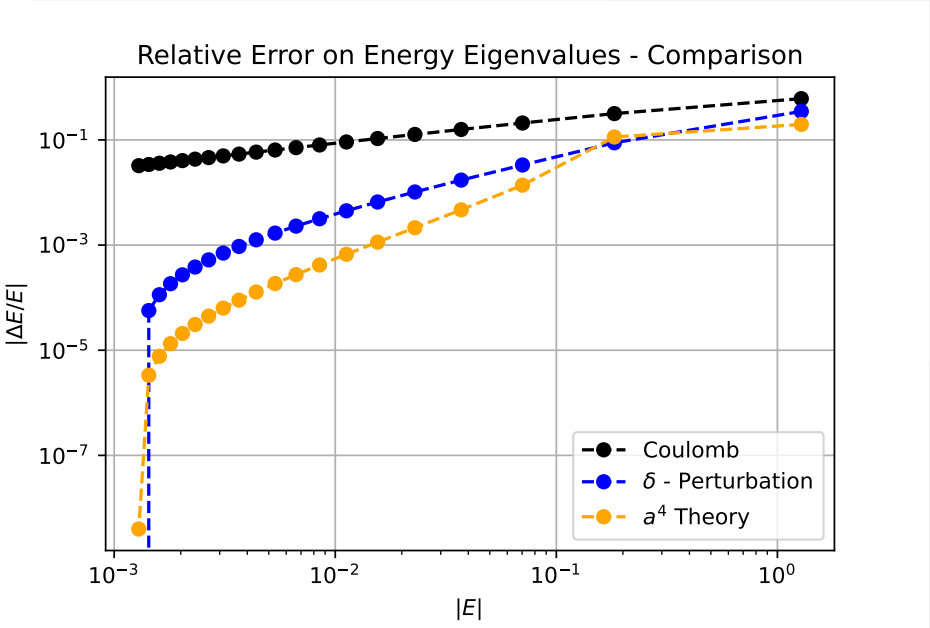
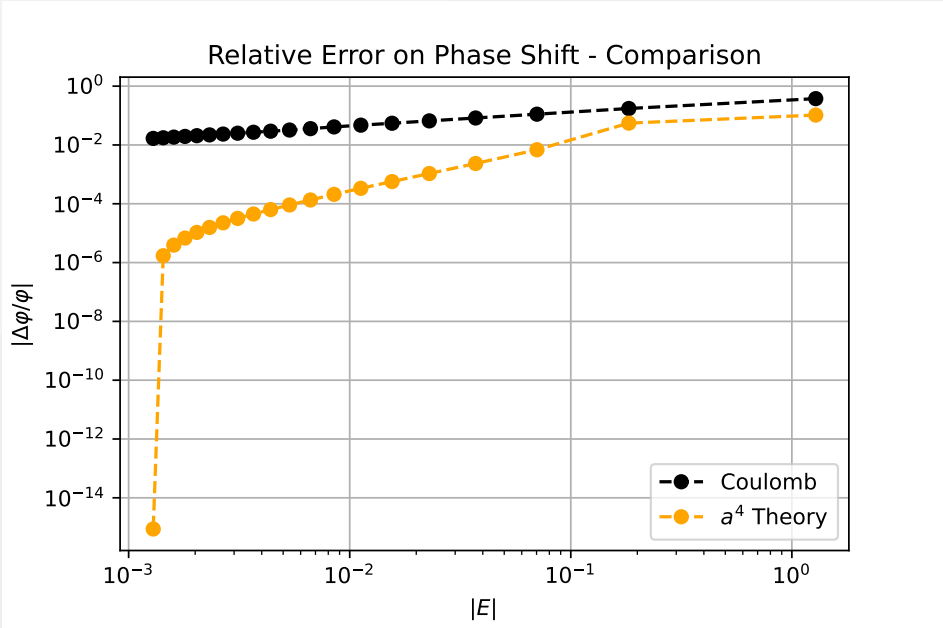


Figure: The accuracy also depends on the size of the cutoff. The results worsen taking higher values for a because we are excluding more momenta. To cancel the dependence on a one should add all the contact terms containing higher order dreivatives of the smeared delta. The crossings between the curves correspond to a change of sign in ΔE.

Results of the effective theory





The effective theory can reproduce the data of the real potential better than the naive approximation with the δ function.

Bibliography

[1] G. P. Lepage. “How to renormalize the Schrodinger equation”. In: *8th Jorge Andre Swieca Summer School on Nuclear Physics*. Feb. 1997, pp. 135–180. arXiv: nucl-th/9706029.

[2] Paolo Giannozzi. *Numerical Methods in Quantum Mechanics*. Lecture notes. 2021.